Menu

Utilities & Infrastructure

Relevant source files

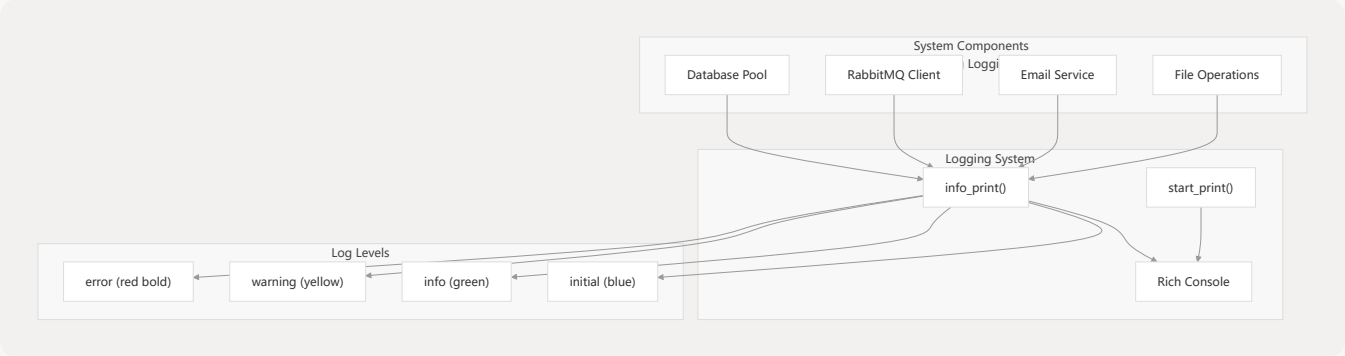
This document covers the core utility functions, infrastructure components, and supporting services that enable the MKTY Smart Healthcare System's backend operations. These utilities provide essential functionality for database management, file processing, security, messaging, and data analysis across the entire system.

For information about the main Flask API endpoints that use these utilities, see Core API Endpoints. For details about the AI model inference services that interact with the message queue infrastructure, see AI & Machine Learning Services.

Logging and System Initialization

The system provides a comprehensive logging infrastructure centered around the `info_print` function and startup display utilities.

Logging Infrastructure



The logging system uses the Rich library to provide colored, formatted output with timestamps. The `start_print` function displays the ASCII art MKTY logo and system information during backend initialization.

Sources: `backend/util.py` `90-134`

Key Logging Functions

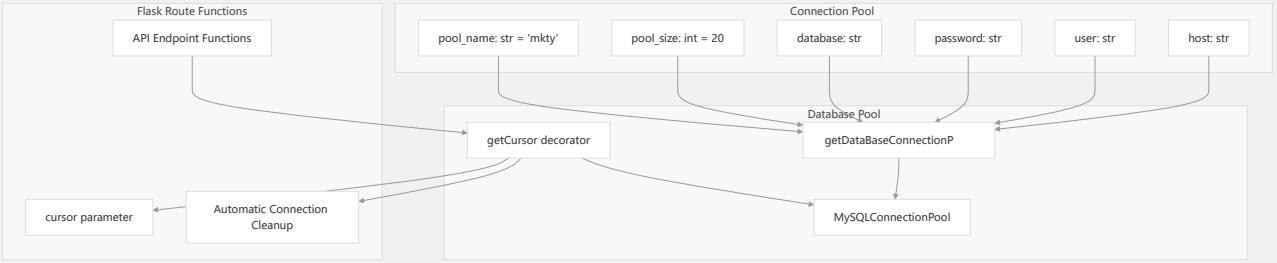
Function	Purpose	Parameters
`info_print`	Print timestamped log messages with color coding	`msg` (str), `level` (str)

Function	Purpose	Parameters
`start_print`	Display system startup banner and information	`version` (str)

Database Infrastructure

The system implements a connection pooling architecture using MySQL Connector/Python to manage database connections efficiently across the Flask application.

Database Connection Management



The `getCursor` decorator automatically manages database connections for Flask route handlers, ensuring proper connection cleanup and error handling.

Sources: `backend/util.py` `190-242`

Database Decorator Usage Pattern

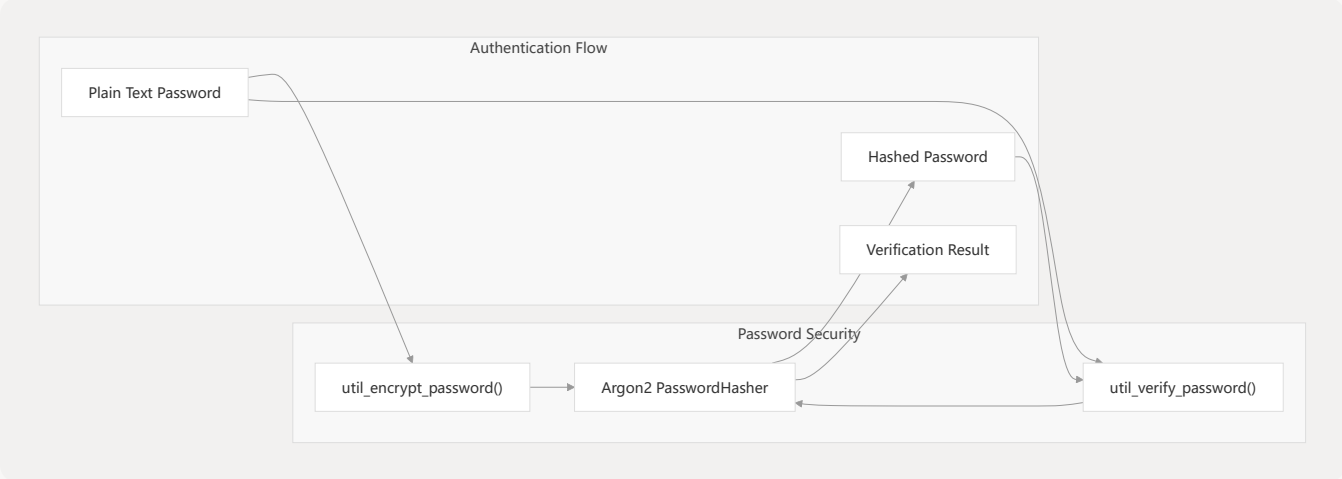
The `getCursor` decorator injects a database cursor into Flask route functions:

```
@getCursor(connection_pool)
def some_api_function(cursor):
    cursor.execute("SELECT * FROM users")
    return cursor.fetchall()
```

Security Infrastructure

The system implements Argon2-based password hashing for secure user authentication.

Password Security Functions



Sources: `backend/util.py` `244-267`

Message Queue Infrastructure

The system uses RabbitMQ for asynchronous communication with AI model services through an RPC (Remote Procedure Call) pattern.

RabbitMQ RPC Client Architecture

The `RpcClient` class manages asynchronous task submission to AI model services and provides methods to check task status and retrieve results.

Sources: `backend/util.py` `269-333`

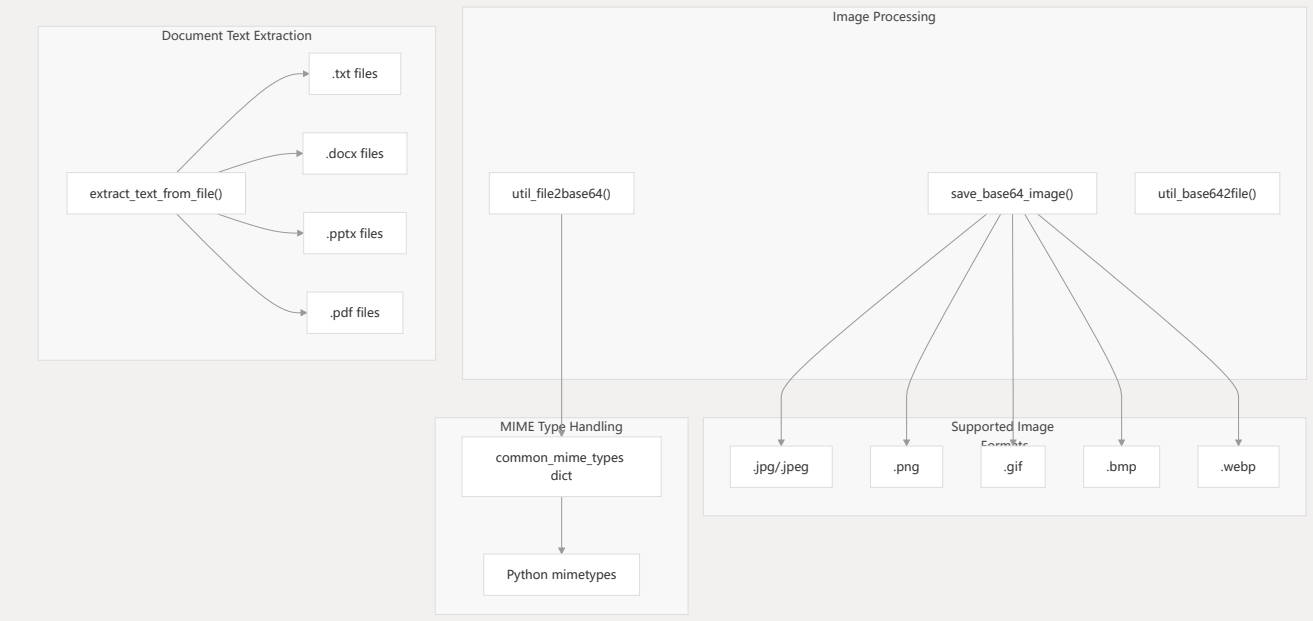
Message Queue Usage Pattern

Method	Purpose	Returns
`call(data)`	Submit task to queue	Task ID (correlation_id)
`get_response(corr_id)`	Check task status and get result	Result data or None

File Processing Utilities

The system provides comprehensive file handling capabilities for various formats including images, documents, and text files.

File Format Support Matrix



Sources: `backend/util.py` `51-86` `backend/util.py` `136-372` `backend/util.py` `578-611`

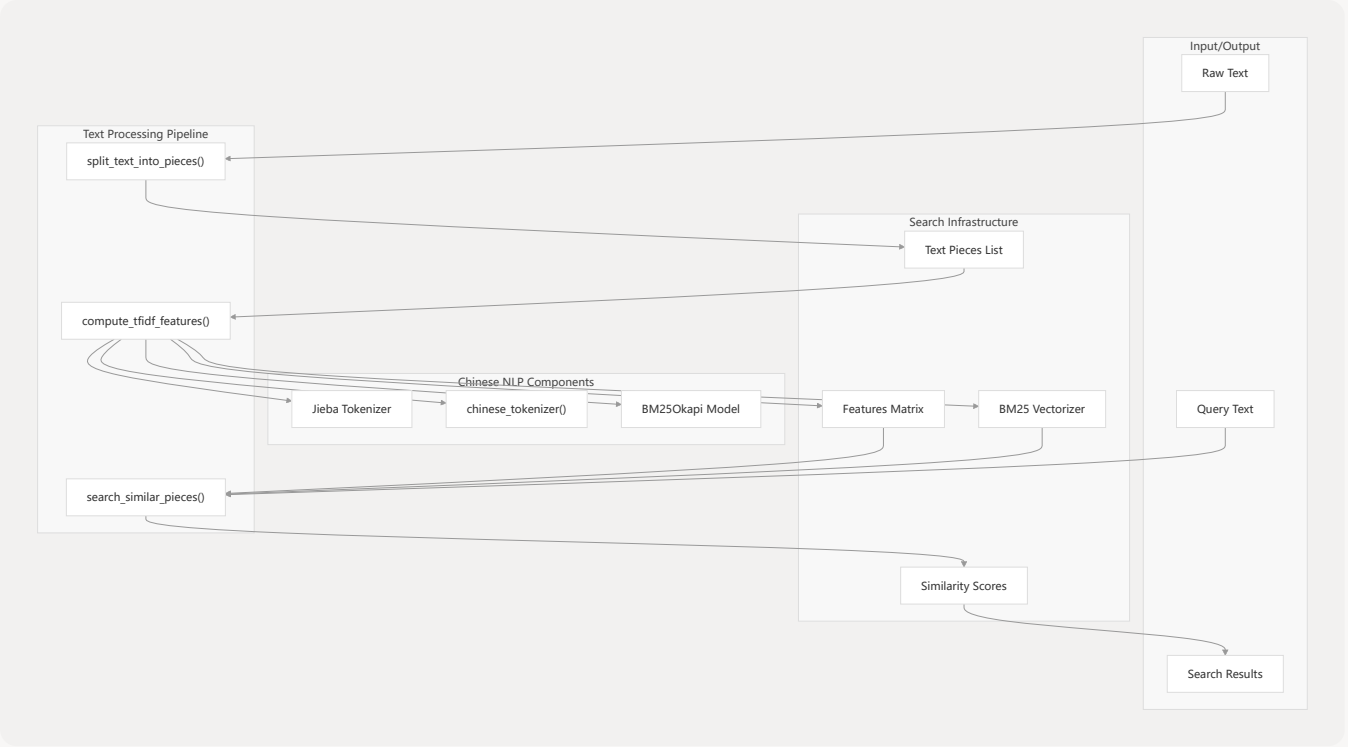
File Processing Functions

Function	Purpose	Input/Output
`util_file2base64`	Convert file to base64 string	File path → Base64 string
`util_base642file`	Convert base64 string to file	Base64 string → File
`save_base64_image`	Save base64 image data efficiently	Base64 → Image file
`extract_text_from_file`	Extract text from documents	Document file → Plain text

Text Processing and Search Infrastructure

The system implements Chinese text processing capabilities using Jieba segmentation and BM25 search algorithms for knowledge retrieval.

Text Analysis Pipeline



Sources: `backend/util.py` `614-719`

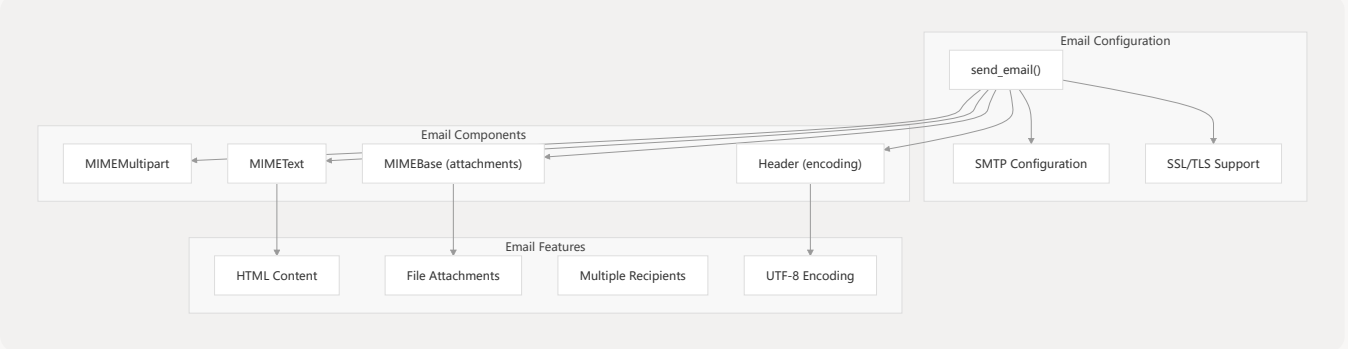
Text Processing Functions

Function	Purpose	Algorithm
`split_text_into_pieces`	Divide text into fixed-length segments	Simple string slicing
`compute_tfidf_features`	Generate BM25 search index	Jieba + BM25Okapi
`search_similar_pieces`	Find relevant text segments	BM25 scoring with normalization

Communication Infrastructure

The system provides email functionality for notifications and document delivery.

Email Service Architecture

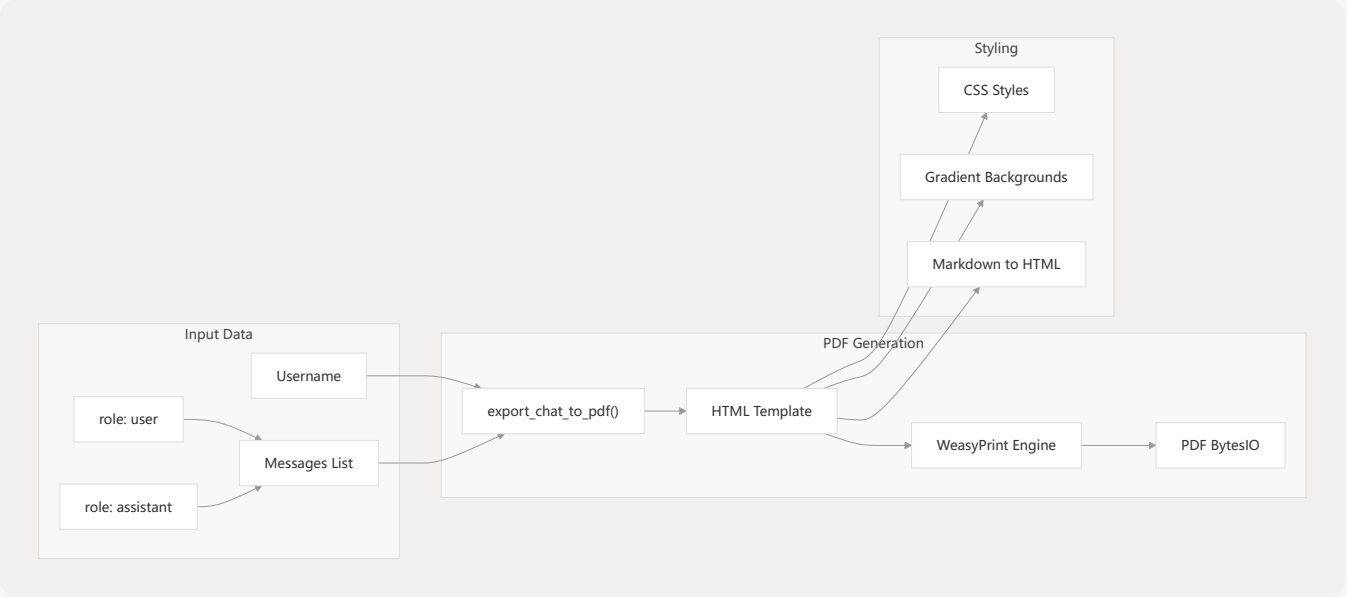


Sources: `backend/util.py` `479-547`

PDF Generation Infrastructure

The system can export chat conversations to formatted PDF documents using HTML-to-PDF conversion.

PDF Export Pipeline

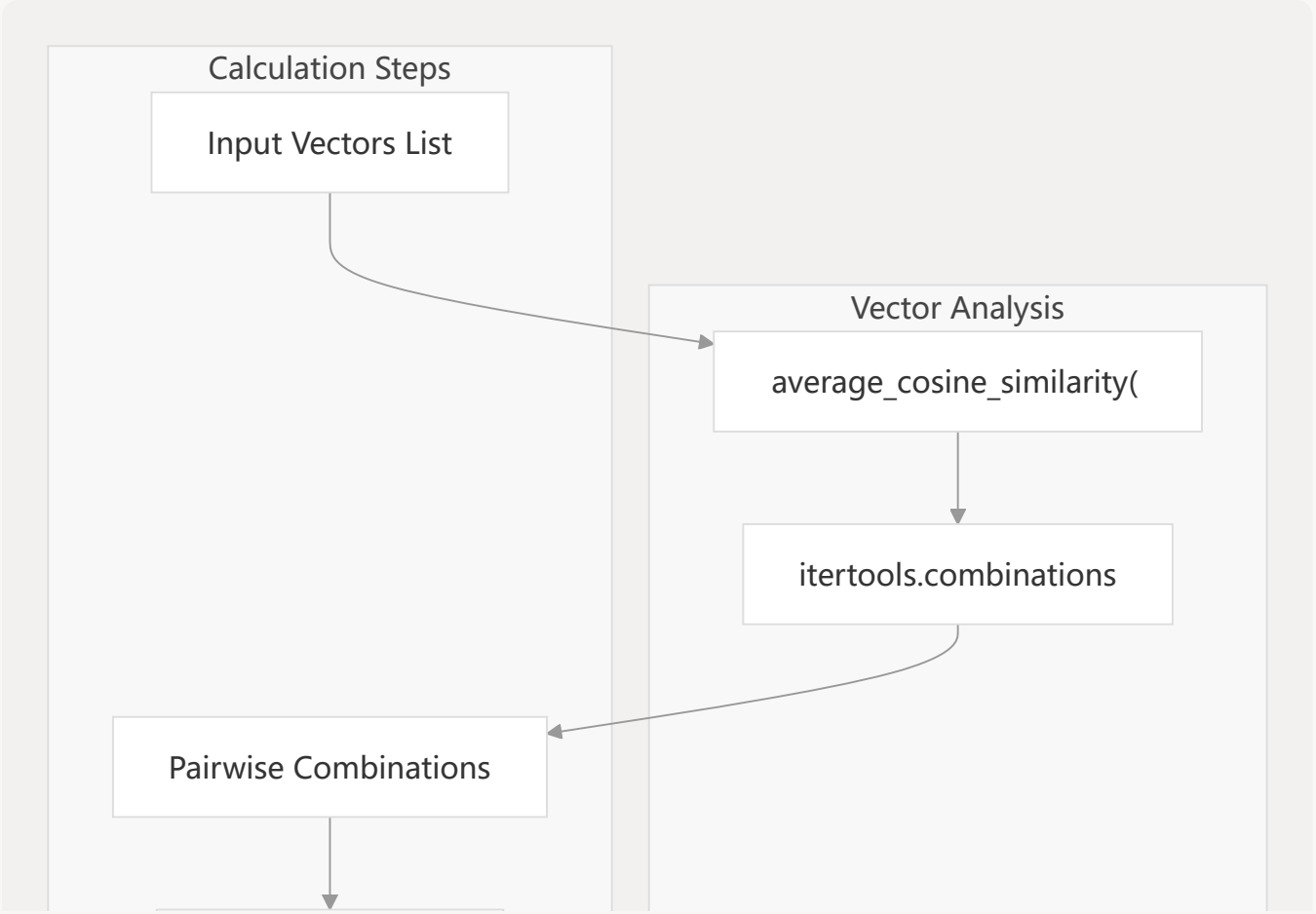


Sources: `backend/util.py` `374-477`

Mathematical and Analysis Utilities

The system includes vector similarity analysis for AI model evaluation and data analysis.

Vector Analysis Functions


Calculation Steps
Input Vectors List
Vector Analysis
average_cosine_similarity(
itertools.combinations
Pairwise Combinations